

Observability Metrics

Observability is an important concept in modern software systems. It helps engineers monitor applications, detect issues quickly, and maintain system reliability.

What is Observability?

Observability is the practice of understanding how a system or application behaves by analysing the data it generates. I Since engineers cannot directly see what is happening inside a system, they analyse the data generated by the system to understand its internal state.

This information is called **telemetry data**, which mainly includes:

- **Metrics**
- **Logs**
- **Traces**

By collecting and analysing this data, teams can monitor system performance, detect problems, and understand how applications behave in real environments.

Why Observability is Important

Modern applications run on complex infrastructures such as cloud platforms, containers, microservices, and distributed systems. Because of this complexity, identifying the root cause of issues becomes difficult.

Observability helps teams:

- Monitor system health.
- Detect failures quickly.
- Understand performance problems.
- Troubleshoot issues faster.
- Improve system reliability.

It also helps organizations make better technical and business decisions based on real operational data.

Core Components of Observability

Observability uses different types of telemetry data that help engineers understand how the system is behaving.

Metrics

Metrics are numerical values collected over time that show the performance and usage of systems.

Examples:

- CPU utilization
- Memory usage
- Response time
- Request rate
- Error count

Metrics help engineers understand **what is happening in the system** and are commonly visualized in monitoring dashboards.

Logs

Logs are records of events that occur within systems, applications, or networks.

Each log entry usually contains:

- Timestamp
- Event details
- System information

Examples:

- Application errors
- User authentication events
- System configuration changes

Logs provide detailed information and help engineers understand **why a problem occurred**.

Traces

Traces track the path of a request as it moves through different services in a distributed system.

For example, a user request may pass through:

- Frontend
- API gateway
- Backend services
- Database

Tracing helps identify slow services or failures and shows **where problems occur in the system**.

Profiles (Emerging Signal)

Profiling analyzes application code to understand how resources like CPU and memory are used.

It helps identify:

- inefficient code
- memory problems
- performance bottlenecks

Because profiling gives deeper insights into application performance, it is considered an emerging pillar of observability.

Types of Observability Metrics

Observability systems usually collect three main types of metrics.

Application Metrics

These measure the performance of applications.

Examples:

- Response time
- Request rate
- Error rate
- Throughput

They help monitor application availability and performance.

System (Infrastructure) Metrics

These measure the health of the underlying infrastructure.

Examples:

- CPU usage
- Memory consumption
- Disk operations
- Network throughput
- Container resource usage

These metrics help teams understand how well system resources are performing.

Business Metrics

Business metrics connect system performance with business outcomes.

Examples:

- Conversion rate
- Number of transactions
- User retention
- Average transaction value

These metrics help organizations understand how technical performance affects business goals.

The Four Golden Signals (SRE Monitoring)

In Site Reliability Engineering (SRE), four key signals are used to measure system health.

Latency

Latency measures how long it takes for a request to be processed.

High latency can cause slow applications and poor user experience.

Traffic

Traffic measures how many requests or transactions the system handles.

It helps understand system usage and scaling requirements.

Errors

Error metrics show how many operations or requests fail.

Monitoring errors helps teams quickly detect issues in applications.

Saturation

Saturation indicates how close system resources are to their limits.

Examples include high CPU usage or memory exhaustion.

High saturation can lead to performance degradation.

Best Practices for Implementing Observability

To implement observability effectively, teams should follow these practices:

Define clear monitoring goals

Organizations should identify the most important metrics instead of monitoring everything.

Use standard telemetry frameworks

Tools such as Open Telemetry help collect telemetry data from different systems in a consistent way.

Automate monitoring and alerts

Automation helps detect issues quickly and reduces manual effort.

Customize dashboards

Custom dashboards help teams focus on the most relevant system metrics.

Conclusion

Observability is essential for managing modern distributed applications and infrastructure. By analysing telemetry data such as metrics, logs, and traces, teams gain visibility into system behaviour and performance.

Monitoring key signals like latency, traffic, errors, and saturation helps maintain reliable and efficient systems. With proper observability practices, organizations can detect issues early, improve performance, and deliver better user experiences.

